

Reviewer Pack — Symmetric Group Fourier Coefficient Gap in Read-Twice BPs fo...

Entry 54da2d60318e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 12:51:12 UTC

Symmetric Group Fourier Coefficient Gap in Read-Twice BPs for IP_2

Entry ID: 54da2d60318e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 12:51:12 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For any read-twice BP P over n variables, let $\rho(P)$ denote the maximum absolute value of the Fourier coefficients of P 's function over the symmetric group S_n . Then, for the IP_2 problem, $\rho(\text{IP}_2 \text{ BP}) = \Omega(n)$, while for all other read-twice BPs, $\rho(P) = O(\log n)$.

Rationale (proposer's reasoning):

The Fourier coefficients over the symmetric group capture the BP's symmetry structure, and the IP_2 problem's inherent symmetry leads to exponentially larger coefficients, distinguishing it from other BPs.

Taxonomy category: BP_READTWICE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 78c5cdccadcff049

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def young_tableaux_characters(n):
        if n == 1:
            return [Fraction(1)]
        chars = [Fraction(1)]
        for i in range(2, n + 1):
            new_chars = []
            k = i - 1
            for j in range(i):
                if j > 0:
                    new_chars.append(chars[j] * (k - j) / j)
                else:
                    new_chars.append(chars[j])
            chars = new_chars
        return chars

    def max_fourier_coefficient(bp, n):
        bp_chars = young_tableaux_characters(n)
        max_coeff = 0
        for char in bp_chars:
            coeff = abs(char * bp[0] + char * bp[1])
            if coeff > max_coeff:
                max_coeff = coeff
        return max_coeff

    def generate_random_bp(n):
        bp = [random.choice([0, 1]) for _ in range(2)]
        return bp

    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""
```

```

for n in n_values:
    for _ in range(5): # Test each n with 5 different BPs
        bp = generate_random_bp(n)
        metric_value = max_fourier_coefficient(bp, n)
        total_metric_value += metric_value
        instances_tested += 1

    if n == 2 and bp == [0, 1]:
        conjecture_holds = False
        counterexample = "IP_2 instance with n=2"

mean_metric_value = total_metric_value / instances_tested
support_fraction = Fraction(instances_tested - (instances_tested // 5), instances_tested)

return {
    "metric_name": "rho",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30)) # Default to 1

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = Fraction(sum(1 for r in results if r["conjecture_holds"]), len(results))

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= Fraction(4, 5): # At least 80%
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='{counterexample}' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c90b4e33.py", line 83, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c90b4e33.py", line 58, in run_trial
    metric_value = max_fourier_coefficient(bp, n)
                    ~~~~~^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c90b4e33.py", line 37, in max_fourier_c
    bp_chars = young_tableaux_characters(n)
                ~~~~~^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c90b4e33.py", line 30, in young_tableau
    new_chars.append(chars[j] * (k - j) / j)
                      ~~~~~^

```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError, preventing evaluation of the conjecture’s validity | next: Fix the young_tableaux_characters implementation to handle index ranges properly

11. Audit log (LLM calls)

Total LLM calls: 13

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	56055
2	propose	ollama_remote	qwen3:8b	0	0	102434
3	propose	ollama_remote	qwen3:8b	0	0	79337
4	propose	ollama_remote	qwen3:8b	0	0	138279
5	propose	ollama_remote	qwen3:8b	0	0	139838
6	preregistration	ollama_remote	qwen3:8b	0	0	32285
7	novelty	ollama_remote	qwen3:8b	0	0	27448
8	novelty	ollama_remote	qwen3:8b	0	0	20091
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17991
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9247
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10712
12	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9067
13	judge	ollama_remote	qwen3:8b	0	0	55207

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 697993 ms total latency. Provider mix: {'ollama_remote': 13}

(full prompt+response transcripts available in research/audit/54da2d60318e.jsonl)

12. Reproducibility

To reproduce this cycle:

- 1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/54da2d60318e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/54da2d60318e.tar.gz` (if generated)